ТЕХНИЧЕСКОЕ ЗАДАНИЕ

(только для студентов на ИУП и своем месте практики)

на учебную практику УП.11 «Разработка, администрирование и защита баз данных»
Срок сдачи работы: до 30.10.2026

! По итогам выполнения добавить ссылку на проект в таблицу(лист УП.11) :
https://docs.google.com/spreadsheets/d/1zc6N10GloXlhYPfE5u8kSPfBt0HDkcuYwxla2mh5dE8/edit?usp=sharing

Описание предметной области

Торговая компания «Чудо Обувь» занимается продажей обуви и имеет сеть магазинов. Приложение для заказа обуви дает возможность клиентам компании выбрать и заказать одну или несколько пар обуви из каталога различных моделей и размеров обуви в торговом зале. На данный момент услуга предоставляется только для обуви, в дальнейшем возможно расширение ассортимента товаров за счет аксессуаров.

Функциональные требования:

- В каталоге должно храниться актуальное доступное для заказа количество товарных позиций, то есть сочетаний модели и размера.
- Для каждой модели в каталоге должно сохраняться изображение.
- Пользователь должен иметь возможность выбрать из каталога одну или несколько пар обуви доступного размера, модели.

- В интерфейсе должен быть доступен просмотр списка заказов с отображением ключевой информации: дата заказа, ФИО клиента;
- Должно быть реализовано управление заказами.

Вход в систему следует производить только по логину, указанному в списке пользователей системы (в ресурсах). Пароль при этом вводить не требуется.

После успешной авторизации пользователь получает доступ к соответствующим функциям системы.

В данной области определены следующие основные составляющие:

Пользователи системы: администратор, менеджер, авторизованный пользователь, неавторизованный пользователь.

Пользователь: логин, фамилия, имя, отчество.

Товары: категория, подкатегория, производство, наименование, изображение, состав, описание, размерный ряд, доступные для заказа размеры, количество, цена.

Товарная позиция: модель, размер.

Заказ: дата заказа, фамилия, имя, отчество клиента, товарная позиция, количество, цена за единицу товара, итоговая сумма по заказу.

Неавторизованный пользователь должен иметь возможность просматривать каталог товаров (допустимо без фильтрации, сортировки, поиска).

Авторизованный пользователь должен иметь возможность просматривать каталог товаров (с фильтрацией, сортировкой и поиском), оформлять заказы: добавлять/удалять товарные позиции, изменять количество.

Менеджер должен иметь возможность просматривать каталог товаров (с фильтрацией, сортировкой, поиском), оформлять заказы (добавлять/удалять товарные позиции, изменять количество),

просматривать заказы, состав конкретного заказа, добавлять новые заказы и удалять существующие.

Администратор должен иметь возможность просматривать каталог товаров (с фильтрацией, сортировкой, поиском), оформлять заказы (добавлять/удалять товарные позиции, изменять количество), просматривать заказы, состав конкретного заказа, добавлять новые заказы и удалять существующие. Администратор должен иметь возможность редактировать заказ.

Взаиморасчеты торговой компании с клиентами осуществляются в рублях.

1. Общие сведения

Наименование темы индивидуального задания: «Разработка, администрирование и защита базы данных системы оформления заказа обуви для торговой компании “Чудо Обувь” средствами SQLite и Python в среде VS Code».

Цель работы: разработать базу данных системы оформления заказа обуви, обеспечить её администрирование и защиту, оформить техническую документацию, загрузить готовый проект в новый репозиторий на GitHub и подготовить его к сдаче.

Заказчик: торговая компания «Чудо Обувь», которая продаёт обувь через сеть магазинов и хочет дать клиентам возможность выбирать и заказывать обувь из каталога.

Предметная область: система оформления заказа обуви. Пользователи: администратор, менеджер, авторизованный пользователь, неавторизованный пользователь. Вход выполняется по логину без пароля. Клиенты выбирают одну или несколько пар обуви из каталога, оформляют заказы. Администратор и менеджер управляют заказами.

2. Требования к репозиторию GitHub

Студент создаёт новый репозиторий на GitHub. Название репозитория формируется по шаблону: up11-shoe-store-<фамилия>. Например, up11-shoe-store-ivanov. Все буквы в названии - латиница, строчные.

Репозиторий должен быть публичным или доступным преподавателю по ссылке. В настройках репозитория студент добавляет преподавателя в качестве коллаборатора, если репозиторий приватный.

При создании репозитория студент выбирает опцию Add a README file - это создаст первый файл в репозитории.

После создания репозитория студент клонирует его на локальный компьютер командой git clone и работает в этой папке. При работе в VS Code студент открывает клонированную папку и ведёт всю работу в ней.

Студент должен делать осмысленные коммиты по ходу работы. Минимум один коммит в день практики. Сообщения коммитов должны отражать суть: Day 1: domain analysis and ER diagram, Day 4: database schema created, Day 8: subqueries and window functions. Нельзя использовать неинформативные сообщения: update, fix, changes, commit.

В конце каждого дня студент выполняет команду git push, чтобы изменения попали в удалённый репозиторий. К концу практики в репозитории должно быть не менее 15 коммитов.

Студент должен убедиться, что файл базы данных db/shoe_store.sqlite не попадает в репозиторий. Для этого файл добавляется в .gitignore. При первом запуске проекта база данных создаётся заново командой python src/db_init.py.

3. Что должно быть в репозитории

Репозиторий должен содержать следующие папки и файлы.

Папка docs - вся техническая документация в формате Markdown и диаграммы в формате PDF/PNG.

Папка src - весь исходный код Python.

Папка db - SQL-скрипты и папка для резервных копий.

Папка queries - SQL-запросы.

Файл README.md - описание проекта и инструкция по запуску.

Файл .gitignore - список исключений для Git.

Файл requirements.txt - список зависимостей.

4. Требования к файлу README.md

Файл README.md находится в корне репозитория. Он должен содержать следующие разделы.

Название проекта и краткое описание.

Автор - ФИО, группа, курс.

Стек технологий - Python, SQLite, VS Code, Git.

Инструкция по запуску - последовательность команд для развёртывания проекта с нуля.

Структура проекта - дерево папок с пояснениями.

Список команд - какие скрипты что делают.

Ссылка на репозиторий - на сам себя.

README должен быть оформлен так, чтобы человек, открывший репозиторий впервые, понял, что это за проект и как его запустить.

5. Требования к базе данных

База данных создаётся в SQLite. Файл базы данных создаётся автоматически при запуске скрипта src/db_init.py.

База данных должна содержать не менее семи таблиц, находящихся в третьей нормальной форме.

Таблица roles - справочник ролей. Поля: идентификатор роли (PK), название роли (UNIQUE, NOT NULL).

Таблица users - пользователи системы. Поля: идентификатор пользователя (PK), фамилия, имя, отчество, логин (UNIQUE, NOT NULL), ссылка на роль (FK).

Таблица products - модели обуви. Поля: идентификатор товара (PK), категория, подкатегория, производитель, наименование, путь к изображению, состав, описание, цена.

Таблица sizes - справочник размеров. Поля: идентификатор размера (PK), значение размера.

Таблица stock_items - товарные позиции (сочетания модели и размера). Поля: идентификатор позиции (PK), ссылка на товар (FK), ссылка на размер (FK), доступное количество.

Таблица orders - заказы. Поля: идентификатор заказа (PK), дата заказа, ссылка на пользователя (FK).

Таблица order_items - состав заказов. Поля: идентификатор позиции заказа (PK), ссылка на заказ (FK), ссылка на товарную позицию (FK), количество, цена на момент оформления.

6. Требования к структуре и ограничениям

Все таблицы связаны внешними ключами. При подключении к базе данных выполняется PRAGMA foreign_keys = ON.

Внешние ключи имеют стратегии удаления, отражающие предметную область.

При удалении пользователя с заказами - запрет (ON DELETE RESTRICT).

При удалении роли, используемой пользователями - запрет (ON DELETE RESTRICT).

При удалении товара, у которого есть позиции - запрет (ON DELETE RESTRICT).

При удалении размера, у которого есть позиции - запрет (ON DELETE RESTRICT).

При удалении заказа - каскадное удаление всех его позиций (ON DELETE CASCADE).

При удалении товарной позиции, вошедшей в заказ - запрет (ON DELETE RESTRICT).

Поля имеют ограничения CHECK, где это логично. Цена больше нуля. Количество не отрицательно. Длина логина от трёх до двадцати символов. Размер обуви в разумном диапазоне.

Поля, которые обязательны, имеют ограничение NOT NULL. Поля, которые уникальны, имеют ограничение UNIQUE.

Для ускорения поиска созданы индексы на все внешние ключи и на поля, по которым выполняется фильтрация и сортировка.

7. Требования к именованию

Все имена таблиц и полей соответствуют требованиям демонстрационного экзамена.

Только латиница в нижнем регистре. Никакой кириллицы.

Только строчные буквы, цифры и символ нижнего подчёркивания.

Имя начинается с буквы.

Имена таблиц во множественном числе: `users`, `products`, `orders`.

Имена полей в единственном числе: `login`, `price`, `quantity`.

Идентификаторы имеют суффикс `_id`: `user_id`, `product_id`, `order_id`.

Нельзя использовать зарезервированные слова SQL: order, select, from, where, index, table, column.

Стиль именования - snake_case: stock_items, order_items, last_name.

8. Требования к типам данных

Типы данных соответствуют назначению полей.

Идентификаторы - INTEGER.

Строковые значения - TEXT.

Даты - TEXT в формате ISO 8601 (YYYY-MM-DD).

Целые числовые значения - INTEGER.

Денежные значения - REAL или NUMERIC.

Типы SQLite сопоставлены с типами из задания демоэкзамена: INTEGER, VARCHAR, DATE, NUMERIC.

9. Требования к импорту данных

База данных заполняется данными из пяти файлов заказчика: Products_import.xlsx, Sizes_import.xlsx, Stock_Items_import.xlsx, Orders_import.xlsx, Users_import.xlsx.

Импорт выполняется скриптом src/import_data.py. Порядок импорта: сначала справочники, потом зависимые таблицы.

После импорта проверена ссылочная целостность. Количество записей в каждой таблице соответствует исходным файлам.

10. Требования к SQL-запросам

Студент пишет не менее 40 SQL-запросов.

Базовые запросы (15 штук) - SELECT, WHERE, ORDER BY, LIMIT, DISTINCT. Сохраняются в queries/queries_day6.sql.

JOIN и агрегаты (15 штук) - INNER JOIN, LEFT JOIN, GROUP BY, HAVING, COUNT, SUM, AVG. Сохраняются в queries/queries_day7.sql.

Подзапросы и оконные функции (10 штук) - EXISTS, IN, ROW_NUMBER, RANK, PARTITION BY. Сохраняются в queries/queries_day8.sql.

Запросы осмысленные и покрывают типовые задачи: список товаров, остатки по размерам, заказы за период, топ клиентов по сумме заказов, товары без заказов.

11. Требования к объектам базы данных

Представления. Не менее трёх представлений: список заказов с ФИО клиента, товары с остатками, топ клиентов. Сохраняются в db/views_indexes.sql.

Индексы. Не менее пяти индексов на часто используемые поля.

Пользовательские функции. Не менее трёх функций, зарегистрированных в SQLite через create_function. Сохраняются в src/procedures_functions.py.

Триггеры. Не менее трёх триггеров: логирование изменений заказов, автоматическое обновление остатков, запрет удаления пользователя с заказами. Сохраняются в db/triggers.sql.

Таблица аудита. Таблица audit_log для записи изменений.

12. Требования к администрированию

Роли и права. На уровне приложения реализованы роли: администратор, менеджер, пользователь, читатель. Каждая роль имеет свой набор прав. Проверка прав выполняется в Python-коде.

Резервное копирование. Скрипт src/backup.py создаёт резервную копию базы данных в папке db/backups с датой в имени файла.

Восстановление. Есть возможность восстановить базу из резервной копии.

Мониторинг. Скрипт src/db_analyze.py собирает статистику: количество записей в каждой таблице, размер базы, список индексов.

13. Требования к защите

Параметризованные запросы. Все запросы, принимающие данные от пользователя, параметризованы. Нельзя использовать f-строки для подстановки значений в SQL.

Хэширование паролей. Если в системе есть пароли, они хранятся в виде хэша SHA-256.

Аудит доступа. Все действия пользователей записываются в журнал.

Разграничение прав. Каждая роль имеет минимально необходимый набор прав.

Политика безопасности. Документ docs/13_security_policy.md с описанием политики.

Демонстрация защиты. Документ docs/13_injection_demo.md с демонстрацией уязвимого и защищённого запроса.

14. Требования к документации

В папке docs должны быть следующие файлы.

00_domain_description.md - текст описания предметной области.

01_entities.md - список сущностей, атрибутов, связей и ограничений целостности.

01_normalization.md - проверка каждой сущности на 1НФ, 2НФ, 3НФ.

01_schema_analysis.md - разбор шести вариантов схем из демоэкзамена.

01_erd_description.md - описание ER-диаграммы.

01_requirements_check.md - проверка покрытия функциональных требований.

01_glossary.md - словарь терминов.

02_data_types.md - таблица типов данных для всех таблиц.

02_type_mapping.md - сопоставление типов SQLite и типов из демоэкзамена.

02_naming_check.md - проверка схемы именования.

02_constraints.md - описание ограничений.

02_schema_report.md - описание созданной структуры.

03_naming_final.md - финальная проверка схемы именования.

03_constraints_final.md - финальный набор ограничений.

03_data_integrity.md - результаты проверки целостности.

03_indexes.md - описание индексов.

03_ddl_report.md - отчёт о DDL-скрипте.

04_schema_report.md - описание финальной структуры.

05_import_report.md - отчёт об импорте.

06_query_results.md - результаты базовых запросов.

07_query_results.md - результаты JOIN и агрегатов.

08_query_results.md - результаты подзапросов и оконных функций.

09_explain_analysis.md - анализ плана выполнения запросов.

10_functions_description.md - описание пользовательских функций.

11_triggers_description.md - описание триггеров.

12_admin_report.md - отчёт по администрированию.

13_security_policy.md - политика безопасности.

13_injection_demo.md - демонстрация защиты от SQL-инъекций.

14_mock_exam_results.md - результаты репетиции демоэкзамена.

15_practice_report.md - итоговый отчёт по практике.

15_self_assessment.md - самооценка.

Дополнительно в docs должны быть файлы ERD_module_1.pdf (ER-диаграмма в PDF) и ERD_module_1.drawio (исходник диаграммы).

15. Требования к исходному коду

Код разбит на модули, каждый не длиннее 300 строк.

src/db_init.py - создание базы данных и всех таблиц.

src/import_data.py - импорт данных из файлов заказчика.

src/db_seed.py - заполнение тестовыми данными.

src/db_test_constraints.py - проверка ограничений.

src/db_naming_check.py - проверка схемы именования.

src/procedures_functions.py - пользовательские функции.

src/backup.py - резервное копирование.

src/db_analyze.py - анализ базы.

src/db_validate.py - проверка данных на аномалии.

src/shell.py - консольный интерфейс для работы с базой.

Каждая функция имеет docstring. Обработка ошибок через try/except. Имена переменных и функций - snake_case.

16. Требования к сдаче

Студент сдаёт:

1. Ссылку на новый репозиторий GitHub.
2. Работающую базу данных SQLite (создаётся командой `python src/db_init.py`).
3. ER-диаграмму в формате PDF.
4. Все SQL-скрипты и Python-скрипты.
5. Полный комплект документации.
6. Файл `README.md` с инструкцией по запуску.
7. Файл `15_practice_report.md` с итоговым отчётом.

17. Критерии оценивания

Критерии оценивания учебной практики УП.11

«Разработка, администрирование и защита баз данных»

№	Критерий	Что проверяется	Баллы
1	Репозиторий GitHub	Новый репозиторий создан с правильным названием, README оформлен, минимум 15 осмысленных коммитов, .gitignore настроен, файл базы данных не попадает в репозиторий	1
2	ER-диаграмма	Диаграмма содержит семь таблиц, все ключи, связи с кардинальностью, соответствует третьей нормальной форме, экспортирована в PDF с именем ERD_module_1.pdf	1
3	Схема именования	Все таблицы и поля названы латиницей в нижнем регистре, snake_case, множественное число для таблиц, суффикс _id для идентификаторов, нет зарезервированных слов	1
4	Типы данных	Типы соответствуют назначению полей, сопоставлены с типами демоэкзамена (INTEGER, VARCHAR, DATE, NUMERIC), обоснованы в документации	1
5	Структура базы данных	Семь таблиц, все первичные и внешние ключи, ограничения NOT NULL, UNIQUE, CHECK, DEFAULT, PRAGMA foreign_keys = ON, стратегии ON DELETE	1
6	Импорт данных	Данные из пяти файлов заказчика загружены, ссылочная целостность проверена, количество записей соответствует исходным файлам	1
7	SQL-запросы	40+ запросов: базовые, JOIN, GROUP BY, подзапросы, оконные функции. Все работают, покрывают типовые задачи системы	1
8	Объекты базы данных	Представления, индексы на внешние ключи, пользовательские функции, триггеры, таблица audit_log. Всё работает	1
9	Администрирование	Роли и права на уровне приложения, резервное копирование, восстановление, мониторинг состояния базы	1
10	Защита базы данных	Параметризация запросов, хэширование паролей, аудит доступа, политика безопасности, демонстрация защиты от SQL-инъекций	0.5
11	Документация	Полный комплект файлов в docs: описание предметной области, нормализация, типы данных, отчёты по дням, протоколы тестирования	0.5
		Итого	10

Максимальная оценка - 10 баллов. Итоговая оценка выставляется по шкале: 9–10 - «отлично», 7–8 - «хорошо», 5–6 - «удовлетворительно», 0–4 - «неудовлетворительно»